ユーザー管理画面UI設計書

メタデータ

title: ユーザー管理画面UI設計書
category: UI設計
version: 1.0.0
last_updated: 2026-02-24
author: Development Team
tags: [system-admin, user-management, crud, csv]

関連ドキュメント

- ユーザー管理API仕様
- データベース設計
- 認証API仕様
- UIポリシー
- セキュリティ規約

---

1. 概要

1.1. 目的

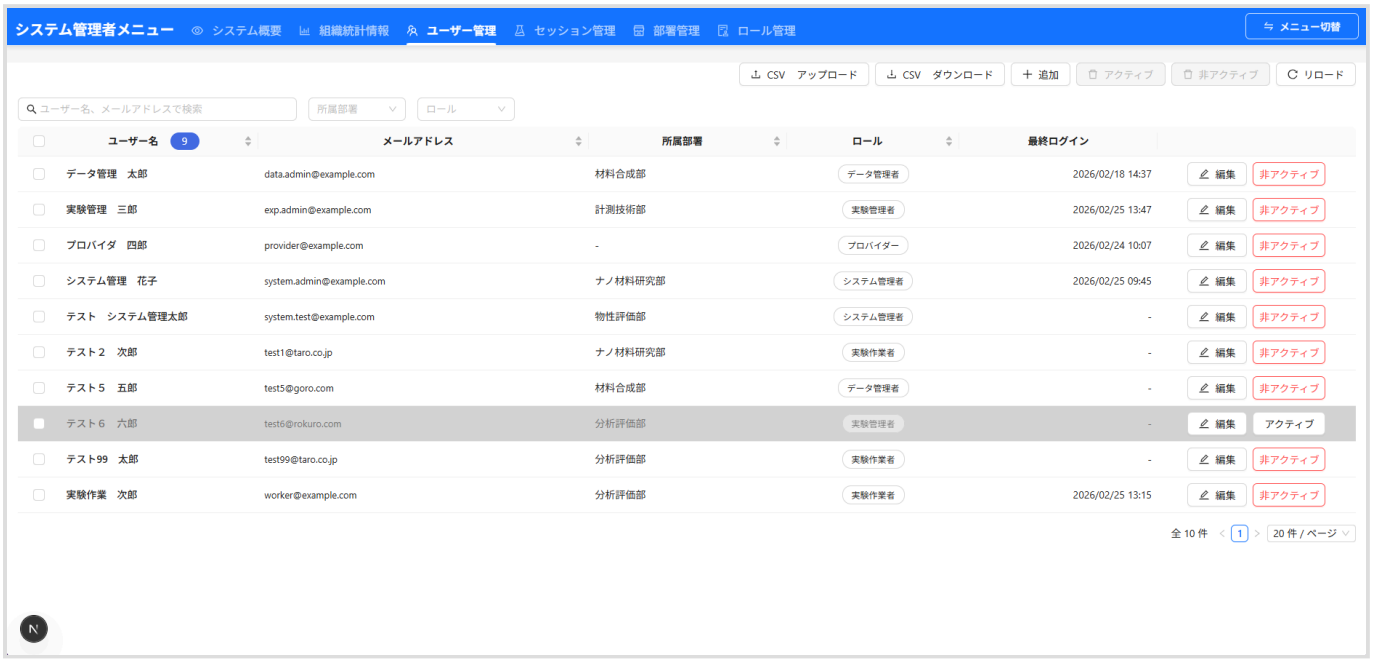
システム管理者が組織内のユーザー（組織利用者）を管理するための画面です。ユーザーの一覧表示、検索、フィルタリング、CRUD操作、CSV操作を提供します。

1.2. 表示条件

- システム管理者メニューから「ユーザー管理」タブを選択

- System Admin権限が必要

1.3. 画面イメージ



2. 画面構成

2.1. レイアウト

```
┌──────────────────────────────────────────────────────┐
│ システム管理者メニュー                                │
│ [ロール管理] [部署管理] [ユーザー管理] [その他]       │
├──────────────────────────────────────────────────────┤
│                                                      │
│  [CSV↑] [CSV↓] [追加] [アクティブ] [非アクティブ] [リロード]  │
│                                                      │
│  ┌──────────────────────────────────────────────┐    │
│  │ 🔍 ユーザー名、メールアドレスで検索          │    │
│  └──────────────────────────────────────────────┘    │
│  [所属部署▼] [二段階認証▼] [ロール▼]                 │
│                                                      │
│  ┌──────────────────────────────────────────────┐    │
│  │□|ユーザー名（8）|メールアドレス      |所属部署 |...|操作|    │
│  ├┼──────────┼──────────────┼────────┼──┼──┤    │
│  │□|山田 太郎      |user001@example...|開発部   |...|編集|    │
│  │□|佐藤 花子      |user002@example...|営業部   |...|編集|    │
```

```
|  |□|鈴木 一郎       |user003@example...|開発部    |...|編集|    |
|  └──────────────────────────────────────────────────────┘    |
|                                                              |
|  全 25 件                          [< 1 2 3 >] 10▼ 件/ページ  |
|                                                              |
└──────────────────────────────────────────────────────────────┘
```

2.2. コンポーネント構成

ツールバー（上部）

- **配置**: 画面上部、右寄せ
- **ボタン**: CSV操作、追加、アクティブ/非アクティブ切替、リロード

フィルターエリア（中段）

- **配置**: ツールバー直下
- **コンポーネント**: 検索入力、3つのSelectフィルター

テーブルエリア（メイン）

- **配置**: 画面中央
- **コンポーネント**: Ant Design `<Table>`

---

3. ツールバー

3.1. CSVアップロードボタン

コンポーネント: `<Button>`

仕様:

項目	値
ラベル	「CSV アップロード」（2行表示）

アイコン	`<UploadOutlined />`
無効化条件	loading=true

動作:

1. クリック時にfile inputを動的に生成
2. `.csv` ファイルのみ受付
3. ファイル選択後、即座にAPI呼び出し
4. 成功時: データ再読み込み + 成功メッセージ
5. 失敗時: エラーメッセージ表示

実装例:

```
const handleUpload = () => {
  const input = document.createElement('input');
  input.type = 'file';
  input.accept = '.csv';
  input.onchange = async (e: Event) => {
    const target = e.target as HTMLInputElement;
    const file = target.files?.[0];
    if (file) {
      try {
        setLoading(true);
        await uploadCsv(file);
        await loadUsers();
        message.success('CSVファイルをアップロードしました');
      } catch (error) {
        message.error(error.message);
      } finally {
        setLoading(false);
      }
    }
  };
  input.click();
};
```

3.2. CSVダウンロードボタン

コンポーネント: `<Button>`

仕様:

項目	値
ラベル	「CSV ダウンロード」（2行表示）
アイコン	`<DownloadOutlined />`
無効化条件	loading=true

動作:

1. 現在のフィルター条件を含めてCSVダウンロードAPIを呼び出し
2. サーバーから返されたBlobをダウンロード
3. ファイル名は `Content-Disposition` ヘッダーから取得
4. フォーマット: `users_YYYYMMDDHHmmss.csv`

3.3. 追加ボタン

コンポーネント: `<Button>`

仕様:

項目	値
ラベル	「追加」
アイコン	`<PlusOutlined />`
無効化条件	loading=true

動作:

1. ユーザー追加ダイアログを開く
2. フォームをリセット
3. 初期値を設定（isActive=true, twoFactorAuth=0）

3.4. アクティブボタン

コンポーネント: `<Button>`

仕様:

項目	値

ラベル	「アクティブ」
アイコン	`<DeleteOutlined />`
無効化条件	選択行なし または loading=true
スタイル	選択時: 背景色=#4CAF50（緑）、文字色=白

動作:

1. 選択されたユーザーをアクティブ化
2. 確認モーダル表示:「選択したユーザーをアクティブにしますか？」
3. OK時: `batchUpdateUserStatus(userIds, true)` API呼び出し
4. 成功時: データ再読み込み + 選択解除 + 成功メッセージ

3.5. 非アクティブボタン

コンポーネント: `<Button>`

仕様:

項目	値
ラベル	「非アクティブ」
アイコン	`<DeleteOutlined />`
タイプ	`danger`
無効化条件	選択行なし または loading=true
スタイル	選択時: 背景色=#FFA500（オレンジ）、文字色=白

動作:

1. 選択されたユーザーを非アクティブ化
2. 確認モーダル表示:「選択したユーザーを非アクティブにしますか？」
3. OK時: `batchUpdateUserStatus(userIds, false)` API呼び出し
4. 成功時: データ再読み込み + 選択解除 + 成功メッセージ

3.6. リロードボタン

コンポーネント: `<Button>`

仕様:

項目	値
ラベル	「リロード」
アイコン	`<ReloadOutlined />`
無効化条件	loading=true

動作:

1. データ再読み込み
2. すべてのフィルター条件をクリア
3. 選択行をクリア
4. 成功メッセージ表示:「データをリロードしました」

---

4. フィルターエリア

4.1. 検索入力

コンポーネント: `<Input>`

仕様:

項目	値
プレースホルダー	「ユーザー名、メールアドレスで検索」
前置アイコン	`<SearchOutlined />`
幅	400px
クリア可能	○
無効化条件	loading=true

検索対象:

- ユーザー名（部分一致、大文字小文字区別なし）
- メールアドレス（部分一致、大文字小文字区別なし）

動作:

- リアルタイムフィルタリング（onChange時）
- クライアントサイドフィルタリング

4.2. 所属部署フィルター

コンポーネント: `<Select>`

仕様:

項目	値
プレースホルダー	「所属部署」
幅	150px
クリア可能	○
無効化条件	loading=true
データソース	departmentApiService.fetchMasters()

選択肢:

```
[
  { value: 'DEPT001', label: '開発部' },
  { value: 'DEPT002', label: '営業部' },
  // ...
]
```

動作:

- department_cdで完全一致フィルタリング
- クライアントサイドフィルタリング

4.3. 二段階認証フィルター

コンポーネント: `<Select>`

仕様:

項目	値
プレースホルダー	「二段階認証」

幅	150px
クリア可能	○
無効化条件	loading=true

選択肢:

```
[
  { value: '済', label: '済' },
  { value: '未', label: '未' }
]
```

動作:

- twoFactorAuthフィールドでフィルタリング（true/false）
- クライアントサイドフィルタリング

4.4. ロールフィルター

コンポーネント: `<Select>`

仕様:

項目	値
プレースホルダー	「ロール」
幅	150px
クリア可能	○
無効化条件	loading=true
データソース	roleApiService.fetchMasters()

選択肢:

```
[
  { value: 'ROLE001', label: 'プロバイダー' },
  { value: 'ROLE002', label: 'システム管理者' },
  // ...
]
```

動作:

- role_cdで完全一致フィルタリング
- クライアントサイドフィルタリング

---

5. ユーザー一覧テーブル

5.1. テーブル仕様

コンポーネント: `<Table>`

仕様:

項目	値
サイズ	`small`
行選択	チェックボックスあり
スクロール	横スクロール有効（x: 1400）
ページング	あり（10/20/50/100件）
ローディング	データ取得中は表示
rowKey	`userId`

5.2. カラム定義

カラム名	データ項目	幅	配置	ソート	備考
（チェックボックス）	-	-	center	×	複数選択用
ユーザー名	`userName`	150px	left	○	バッジ表示あり
メールアドレス	`mailAddress`	250px	left	○	-
所属部署	`departmentName`	150px	left	○	JOINで取得

二段階認証設定	`twoFactorAuth`	130px	center	○	済/未表示
ロール	`roleName`	130px	center	○	Tagで表示
最終ログイン	`lastLoginAt`	150px	right	×	フォーマット済み
操作	-	150px	center	×	編集、アクティブ/非アクティブボタン

5.3. ユーザー名カラム（アクティブユーザー数バッジ）

表示内容:

```
┌──────────────────┐
│ ユーザー名    (8) │
└──────────────────┘
```

バッジ仕様:

- **数値**: フィルタリング後のis_active=trueのユーザー数
- **背景色**: `#4169E1`（ロイヤルブルー）
- **サイズ**: fontSize=14px, height=24px, minWidth=40px
- **形状**: borderRadius=12px（丸型）

実装例:

```
title: (
  <div style={{ textAlign: 'center' }}>
    <Space size="middle" align="center">
      <span>ユーザー名</span>
      <Badge
        count={filteredUsers.filter(user => user.isActive === true).length}
        style={{
          backgroundColor: '#4169E1',
          fontSize: 14,
          height: 24,
          minWidth: 40,
          lineHeight: '24px',
          borderRadius: 12,
        }}
      />
```

```
    </Space>
  </div>
),
```

5.4. 最終ログイン カラム

データソース: login_historyテーブルからMAX(login_at)を取得（success=trueのみ）

表示フォーマット:

- 未ログイン: `ー`
- ログイン済み: `YYYY/MM/DD HH:mm`
- 例: `2026/02/24 14:33`

実装例:

```
render: (value: string | null, record) => {
  if (!value) {
    return <Text>-</Text>;
  }
  // ISO日時をフォーマット
  const date = new Date(value);
  const formatted = `${date.getFullYear()}/${String(date.getMonth() + 1).padStart(2,
'0')}/${String(date.getDate()).padStart(2, '0')} ${String(date.getHours()).padStart(2,
'0')}:${String(date.getMinutes()).padStart(2, '0')}`;
  return <Text>{formatted}</Text>;
}
```

5.5. 操作カラム

コンポーネント: `<Space>` + `<Button>` × 2

編集ボタン:

- ラベル: 「編集」
- アイコン: `<EditOutlined />`
- 動作: ユーザー編集ダイアログを開く

アクティブ/非アクティブボタン:

- ラベル: 現在の状態により切り替え
  - is_active=true → 「非アクティブ」（danger）
  - is_active=false → 「アクティブ」（default）
- 幅: 100px
- 動作: 確認モーダル表示後、ステータス更新API呼び出し

5.6. 行の背景色制御

CSSクラス:

```
.active-row td {
  background-color: #fff !important;
}
.active-row:hover td {
  background-color: #f5f5f5 !important;
}
.inactive-row td {
  background-color: #d3d3d3 !important; /* 明るいグレー */
}
.inactive-row:hover td {
  background-color: #c0c0c0 !important; /* 少し濃いグレー */
}
```

適用:

```
rowClassName={(record) => (record.isActive ? 'active-row' : 'inactive-row')}
```

5.7. ソート機能

ソート可能カラム:

- ユーザー名
- メールアドレス
- 所属部署
- 二段階認証設定

- ロール

ソート方式: クライアントサイドソート（Ant Design標準）

実装例:

```
sorter: (a, b) => (String(a.userName) || '').localeCompare(String(b.userName) || ''),
```

---

6. ページング

6.1. ページング仕様

コンポーネント: `<Table pagination={...}>`

仕様:

項目	値
デフォルトページサイズ	20件
ページサイズ選択	あり
選択肢	10/20/50/100件
合計件数表示	あり（ 全 ${total} 件 ）
配置	テーブル下部、右寄せ

実装例:

```
pagination={{
  pageSize: 20,
  showSizeChanger: true,
  pageSizeOptions: ['10', '20', '50', '100'],
  showTotal: (total) => `全 ${total} 件`,
}}
```

---

7. データ取得と更新

7.1. 初期データ読み込み

タイミング: コンポーネントマウント時（useEffect）

処理フロー:

```
useEffect(() => {
  loadUsers();       // ユーザー一覧取得
  loadDepartments(); // 部署マスター取得
  loadRoles();       // ロールマスター取得
}, []);
```

7.2. ユーザー一覧取得

API: `GET /api/v1/systemManagement/user`

処理:

1. ローディング状態に変更
2. `fetchUsers()` API呼び出し
3. 成功時: stateに保存
4. 失敗時: エラーメッセージ表示
5. ローディング解除

7.3. 4テーブルJOIN構造

結合テーブル:

1. **organization_user**（主テーブル）
2. **department**（LEFT JOIN: 部署名取得）
3. **role**（LEFT JOIN: ロール名取得）
4. **user_authentication**（LEFT JOIN: 二段階認証フラグ取得）
5. **login_history**（LEFT JOIN: 最終ログイン日時取得）

SQLクエリ例:

```
SELECT
  ou.user_id,
  ou.user_name,
  ou.mail_address,
  ou.department_cd,
  d.department_name,
  ou.role_cd,
  r.role_name,
  ua.entry_flg AS two_factor_auth,
  ou.is_active,
  lh.last_login AS last_login_at,
  ou.create_at,
  ou.update_at
FROM organization_user ou
LEFT JOIN department d ON ...
LEFT JOIN role r ON ...
LEFT JOIN user_authentication ua ON ...
LEFT JOIN (
  SELECT user_id, organization_id, MAX(login_at) as last_login
  FROM login_history
  WHERE success = true
  GROUP BY user_id, organization_id
) lh ON ...
WHERE ou.organization_id = $1;
```

---

8. エラーハンドリング

8.1. エラーメッセージ一覧

エラーコード	メッセージ	表示場所
`AUTH_TOKEN_INVALID`	「認証トークンが無効です」	トースト
`AUTH_INSUFFICIENT_PERMISSIONS`	「権限が不足しています」	トースト
`SYSTEM_DATABASE_ERROR`	「データベースエラーが発生しました」	トースト
`NETWORK_ERROR`	「ネットワークエラーが発生しました」	トースト

8.2. エラー表示方法

トーストメッセージ:

```
message.error('ユーザー一覧の取得に失敗しました');
```

確認モーダル:

```
modal.confirm({
  title: '選択したユーザーを非アクティブにしますか？',
  content: `${selectedRowKeys.length}件のユーザーが非アクティブになります。`,
  okText: '非アクティブ化',
  cancelText: 'キャンセル',
  okButtonProps: { danger: true },
  onOk: async () => {
    // 処理
  },
});
```

---

9. ユーザーフロー

9.1. 正常フロー（検索とフィルタリング）

```
[ユーザー管理画面] 表示
    ↓ データ読み込み（ユーザー、部署、ロール）
[ユーザー一覧テーブル] 表示（全件）
    ↓ 検索入力: "山田"
[フィルタリング] クライアントサイド
    → ユーザー名に"山田"を含むユーザーのみ表示
    ↓ 所属部署フィルター選択: "開発部"
[フィルタリング] さらに絞り込み
    → 開発部のユーザーのみ表示
    ↓ ソートクリック: "ロール"カラム
[ソート] クライアントサイド
    → ロール名の昇順で表示
```

9.2. 正常フロー（一括アクティブ化）

```
[ユーザー一覧テーブル]
    ↓ チェックボックスで複数ユーザー選択（3件）
[アクティブボタン] 活性化（背景色：緑）
    ↓ アクティブボタンクリック
[確認モーダル] 表示
    「選択したユーザーをアクティブにしますか？」
    「3件のユーザーがアクティブになります。」
    ↓ OKクリック
API呼び出し：batchUpdateUserStatus(userIds, true)
    ↓ 成功
[ユーザー一覧] 再読み込み
    選択解除
    成功メッセージ表示："3件のユーザーをアクティブにしました"
```

9.3. エラーフロー（CSV アップロード失敗）

```
[CSVアップロードボタン] クリック
    ↓ ファイル選択ダイアログ表示
[ファイル選択] users.csv
    ↓ ファイル選択
API呼び出し：uploadCsv(file)
    ↓ エラー（バリデーションエラー）
[ユーザー管理画面]
    エラーメッセージ表示："2行目：メールアドレスの形式が不正です"
```

---

10. レスポンシブデザイン

10.1. ブレークポイント

サイズ	幅	テーブル幅	フィルター配置
xs	< 576px	100%	縦並び
sm	576px ~ 768px	100%	縦並び
md	768px ~ 992px	100%	横並び
lg	992px ~ 1200px	100%	横並び

xl	1200px ~ 1600px	100%	横並び
xxl	> 1600px	最大3600px	横並び

10.2. モバイル対応

テーブルスクロール

- 横スクロール有効（scroll={{ x: 1400 }}）
- カラム固定: 操作カラムを右端に固定

ツールバーのレイアウト

- モバイル: 縦並び、全幅ボタン
- デスクトップ: 横並び、アイコン+ラベル

---

11. アクセシビリティ

11.1. キーボード操作

Tab順序:

1. ツールバーボタン（左から右）
2. 検索入力
3. フィルターSelect（所属部署、二段階認証、ロール）
4. テーブル（行選択、編集ボタン）
5. ページング

ショートカットキー:

- Enter: フォーカス中のボタンをクリック
- Space: チェックボックスの切り替え
- Escape: モーダルを閉じる

11.2. スクリーンリーダー対応

ARIA属性:

```
<Input
  placeholder="ユーザー名、メールアドレスで検索"
  prefix={<SearchOutlined />}
  aria-label="ユーザー検索"
  aria-describedby="search-help"
/>

<Table
  aria-label="ユーザー一覧"
  rowKey="userId"
  {...otherProps}
/>
```

---

12. パフォーマンス最適化

12.1. フィルタリングの最適化

メモ化:

```
const filteredUsers = useMemo(() => {
  return users.filter(user => {
    // フィルタリングロジック
  });
}, [users, searchText, selectedDepartment, selectedTwoFactorAuth, selectedRole]);
```

12.2. データ取得の最適化

並列読み込み:

```
useEffect(() => {
  Promise.all([
    loadUsers(),
```

```
    loadDepartments(),
    loadRoles()
  ]);
}, []);
```

---

13. セキュリティ考慮事項

13.1. マルチテナント分離

- すべてのAPI呼び出しに `organization_id` を自動付与（authMiddleware）
- 他組織のユーザーデータは閲覧・操作不可

13.2. 権限チェック

- System Admin権限必須
- authMiddleware + permissionCheckで検証

13.3. 機密情報の保護

- パスワードはAPI応答に含めない
- メールアドレス